

LABORATORIO NO. 2
EVALUACIÓN DE LA SEGMENTACIÓN

PRESENTADO POR:
VIVIANA ANDREA BAUTISTA PULIDO
CARMEN EDILIA RICARDO GELVES
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING MAGDA DEL PILAR SANTA FAJARDO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
03 DE ABRIL
2025

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
DESARROLLO DEL LABORATORIO	4
METODO SPLIT AND MERGE.....	4
METODO GRADIENT VECTOR FLOW (GVF)- FLUJO DE VECTORES DE GRADIENTE	7
METODO WATERSHED (CUENCAS HIDROGRAFICAS).....	10
METODO CRECIMIENTO DE REGIONES BASADO EN GRAFOS (GRAPH-BASED REGION GROWING).....	13
CONCLUSIONES.....	15
BIBLIOGRAFÍA.....	17

INTRODUCCIÓN

La segmentación de imágenes es un proceso fundamental en visión por computadora y procesamiento digital de imágenes, cuyo objetivo es dividir una imagen en regiones significativas para facilitar su análisis e interpretación. Este procedimiento es ampliamente utilizado en diversas aplicaciones como reconocimiento de objetos, análisis médico, monitoreo satelital, automatización industrial y conducción autónoma.

Existen múltiples enfoques para la segmentación de imágenes, cada uno con sus propias características y niveles de precisión. En esta actividad, se exploró e implementó el Crecimiento de Regiones basado en Grafos, utilizando el método de Felzenszwalb, una técnica que modela la imagen como un grafo ponderado donde los píxeles se representan como nodos y los bordes indican la similitud entre ellos. A partir de este modelo, el algoritmo agrupa los píxeles en regiones homogéneas optimizando la diferencia de color y contraste entre los segmentos.

Ahora el propósito de este experimento es aplicar y evaluar la segmentación de imágenes basada en grafos en un entorno realista, comparando los resultados obtenidos con una segmentación de referencia (ground truth). Para ello, se llevaron a cabo los siguientes pasos:

1. Carga y preprocesamiento de la imagen: Conversión a escala de grises y eliminación de ruido.
2. Aplicación del algoritmo Felzenszwalb: Segmentación de la imagen en regiones con coherencia visual.
3. Comparación con el ground truth: Evaluación de la calidad de la segmentación mediante el Índice de Jaccard (IoU).
4. Visualización de resultados: Comparación gráfica entre la imagen original, la segmentación obtenida y la segmentación manual.
5. Análisis y conclusiones: Discusión sobre la efectividad del método, sus ventajas, limitaciones y posibles mejoras.

Es importante precisar de igual manera que consideramos que la calidad de la segmentación depende en gran medida de la correcta configuración de los parámetros del algoritmo, como la escala, el suavizado sigma y el tamaño mínimo de las regiones. Ajustar estos valores correctamente es clave para obtener resultados óptimos y evitar una sobre-segmentación o una segmentación demasiado general.

Ahora, uno de los principales desafíos en segmentación de imágenes es encontrar un equilibrio entre la precisión y la eficiencia computacional. Métodos más sofisticados, como redes neuronales profundas o modelos híbridos, pueden ofrecer segmentaciones más precisas, pero requieren una gran cantidad de datos y poder computacional. En este contexto, el Crecimiento de Regiones basado en Grafos se presenta como una alternativa eficiente y efectiva, especialmente en aplicaciones donde se necesita una segmentación rápida y adaptable.

En cuanto a el desarrollo de esta actividad lo dividimos en secciones detalladas donde se explica cada paso del proceso de segmentación, desde la carga y preprocesamiento de la imagen hasta la evaluación del rendimiento del algoritmo. Para finalmente, presentar las conclusiones y posibles mejoras para optimizar la segmentación en futuras aplicaciones.

Con esta introducción, se establece el marco teórico y práctico de la actividad, proporcionando una base sólida para el análisis de los resultados obtenidos y la discusión de su impacto en aplicaciones del mundo real.

DESARROLLO DEL LABORATORIO

La segmentación de imágenes es un proceso clave en visión por computadora que permite dividir una imagen en regiones homogéneas para identificar objetos de interés. En este trabajo, implementamos y evaluamos cuatro métodos de segmentación: Split and Merge, Gradient Vector Flow (GVF), Watershed y Crecimiento de Regiones basado en Grafos. Cada técnica se aplicó a imágenes de prueba y se comparó con un ground truth, utilizando el Índice de Jaccard (IoU) como métrica de desempeño. A continuación, se presenta el desarrollo de la metodología y los resultados obtenidos.

METODO SPLIT AND MERGE

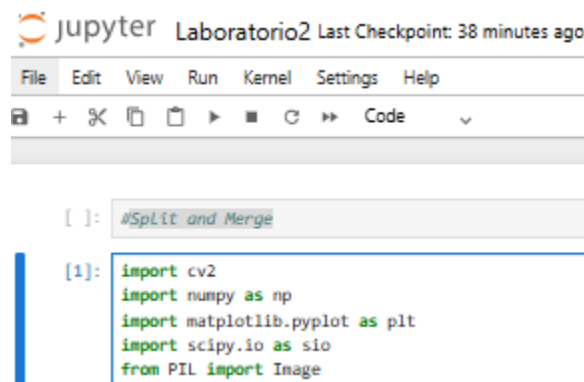
Este código implementa el método Split and Merge para segmentación de imágenes y lo evalúa comparándolo con un ground truth.

Ahora una breve descripción del flujo de la ejecución del código:

- Carga la imagen y su segmentación de referencia (ground truth).
- Convierte la imagen a escala de grises para facilitar el procesamiento.
- Aplica la segmentación Split and Merge dividiendo la imagen en regiones más pequeñas y fusionándolas si son homogéneas.
- Refina la segmentación usando operaciones morfológicas.
- Binariza la imagen segmentada con un umbral adaptativo.
- Calcula el Índice de Jaccard (IoU) para comparar la segmentación obtenida con el ground truth.
- Muestra los resultados incluyendo la imagen original, el ground truth y la segmentación obtenida.

Este enfoque es útil para segmentar regiones con diferentes niveles de intensidad en imágenes complejas. A continuación, procedemos con su explicación en detalle:

1. Lo primero es proceder con la importación de las siguientes librerías:
 - cv2 (OpenCV) → Para lo concerniente al procesamiento de imágenes.
 - numpy → Se basa en las operaciones matriciales.
 - matplotlib.pyplot → Para la visualización de imágenes.
 - scipy.io → Para cargar archivos .mat de MATLAB.
 - PIL (Pillow) → Para la manipulación de imágenes.



```
[ ]: #Split and Merge

[1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
from PIL import Image
```

2. Ahora procedemos con la definición de rutas de imágenes donde definimos las rutas de **img_path** para la imagen de prueba y **gt_path** para el Ground truth donde ejecutamos una segmentación manual.

```
# Rutas de Las imágenes
img_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio N
gt_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio N
```

- Entonces procedemos con la carga de la imagen y el ground truth donde cargamos la imagen con PIL y efectuamos la conversión de RGB a BGR que es el formato OpenCV.

```
# Cargar imagen
image_pil = Image.open(img_path)
image = cv2.cvtColor(np.array(image_pil), cv2.COLOR_RGB2BGR)
```

- Ahora procedemos a cargar el archivo .mat con `scipy.io.loadmat()` y extraemos la segmentación esperada con ground truth.

```
# Cargar el ground truth
mat_data = sio.loadmat(gt_path)
ground_truth = mat_data['groundTruth'][0, 0]['Segmentation'][0, 0]
```

- Ahora procedemos con la conversión a escala de grises donde convertimos la imagen de color a escala de grises lo cual es necesario para la segmentación.

```
# Convertir la imagen a escala de grises
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

- Entonces definimos la función de homogeneidad donde evaluamos si una región es homogénea. Usamos la desviación estándar (`np.std`) como criterio de variabilidad y si la desviación estándar es menor que el umbral (`threshold`), la región es considerada homogénea.

```
def homogeneity_criteria(region, threshold=20):
    return np.std(region) < threshold
```

- Ahora procedemos con la ejecución del algoritmo Split and Merge en donde se obtiene el alto (h) y ancho (w) de la imagen y se inicializa segmented como una imagen negra del mismo tamaño.

```
def split_and_merge(image, min_size=5, threshold=25):
    h, w = image.shape[:2]
    segmented = np.zeros((h, w), dtype=np.uint8)
```

- Ahora se procede a definir la función recursiva `split()` y se extrae una región de la imagen en las coordenadas (x, y) con tamaño (width, height).

```
def split(x, y, width, height):
    region = image[y:y+height, x:x+width]
```

- Entonces si la región es muy pequeña ($\leq \text{min_size}$), se detiene la recursión y se asigna el valor medio de la región a segmented.

```
if width <= min_size or height <= min_size:
    segmented[y:y+height, x:x+width] = np.mean(region)
    return
```

- Por otra parte si la región NO es homogénea, se divide en 4 subregiones y se llama recursivamente a `split()` en cada una de ellas.

```

if not homogeneity_criteria(region, threshold):
    half_w, half_h = width // 2, height // 2
    if half_w > 0 and half_h > 0:
        split(x, y, half_w, half_h)
        split(x + half_w, y, half_w, half_h)
        split(x, y + half_h, half_w, half_h)
        split(x + half_w, y + half_h, half_w, half_h)

```

11. Ahora si la región es homogénea, se asigna su valor promedio a segmented.

```

else:
    segmented[y:y+height, x:x+width] = np.mean(region)

```

12. Y entonces Se inicia la segmentación desde la región completa (0, 0, w, h) y se devuelve la imagen segmentada.

```

split(0, 0, w, h)
return segmented

```

13. Entonces procedemos con la aplicación del método Split and Merge donde se llama a la función con los valores especificados de min_size y threshold.

```

segmented_image = split_and_merge(image_gray, min_size=3, threshold=30)

```

14. Definimos entonces un elemento estructurante elíptico de tamaño 5x5 para mejorar la segmentación.

```

# Aplicar morfología para mejorar la segmentación
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

```

15. Ahora procedemos a rellenar huecos pequeños con Cierre (MORPH_CLOSE) y a eliminar pequeños objetos de ruido con Apertura (MORPH_OPEN). De igual manera a aumentar las regiones segmentadas y suavizar los bordes.

```

segmented_post = cv2.morphologyEx(segmented_image, cv2.MORPH_CLOSE, kernel)
segmented_post = cv2.morphologyEx(segmented_post, cv2.MORPH_OPEN, kernel)
segmented_post = cv2.dilate(segmented_post, kernel, iterations=2)
segmented_post = cv2.erode(segmented_post, kernel, iterations=2)

```

16. Desarrollamos el proceso de Binarización con Otsu donde se aplica umbral adaptativo de Otsu para binarizar la imagen segmentada.

```

# Binarización con umbral de Otsu
segmented_binary = cv2.adaptiveThreshold(segmented_post, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)

```

17. Y ahora convertimos el ground truth a binario (1 para objeto, 0 para fondo).

```

ground_truth_binary = (ground_truth > 0).astype(np.uint8)

```

18. Procedemos entonces con el cálculo del índice de Jaccard (IoU) donde si las imágenes no tienen el mismo tamaño, se ajusta el ground truth.

```

def calculate_iou(segmented, ground_truth):
    if segmented.shape != ground_truth.shape:
        ground_truth_resized = cv2.resize(ground_truth, (segmented.shape[1], segmented.shape[0]), interpolation=cv2.INTER_NEAREST)
    else:
        ground_truth_resized = ground_truth

```

19. Ahora utilizamos Intersección (AND) para píxeles en ambas imágenes. Unión (OR) para píxeles en al menos una imagen y IoU que es la Intersección / Unión.

```
intersection = np.logical_and(segmented, ground_truth_resized)
union = np.logical_or(segmented, ground_truth_resized)
iou_score = np.sum(intersection) / np.sum(union) if np.sum(union) > 0 else 0
```

20. Y procedemos a calcular y almacenar el índice IoU.

```
return iou_score
iou = calculate_iou(segmented_binary, ground_truth_binary)
```

21. Procedemos entonces a mostrar la imagen original, ground truth y la segmentación.

```
# Mostrar imágenes
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axes[0].set_title("Imagen Original")
axes[0].axis("off")

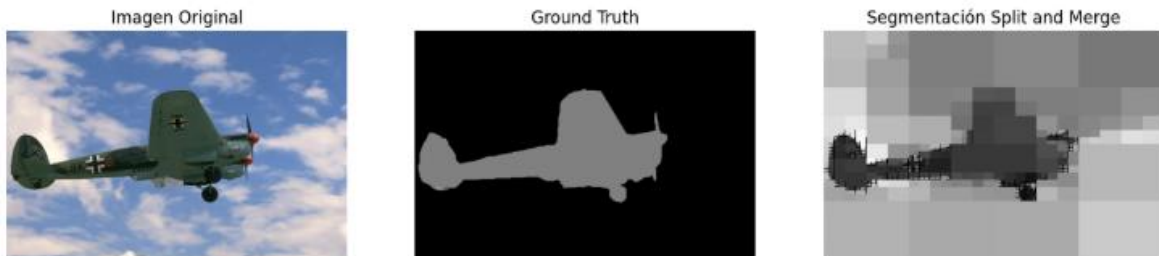
axes[1].imshow(ground_truth, cmap="gray")
axes[1].set_title("Ground Truth")
axes[1].axis("off")

axes[2].imshow(segmented_image, cmap="gray")
axes[2].set_title("Segmentación Split and Merge")
axes[2].axis("off")
```

22. Finalmente procedemos a imprimir el IoU como métrica de evaluación.

```
plt.figtext(0.5, 0.02, f"Índice de Jaccard (IoU): {iou:.4f}", ha="center", fontsize=12)
plt.show()
```

23. A continuación, el resultado de la consola con el desarrollo de la imagen:



Índice de Jaccard (IoU): 0.9003

METODO GRADIENT VECTOR FLOW (GVF)- FLUJO DE VECTORES DE GRADIENTE

Este código implementa un enfoque de segmentación basado en el Campo de Fuerza de Gradiente (GVF), utilizando el operador de Sobel para detectar bordes y mejorar la segmentación. Se compara con una segmentación de referencia (ground truth) y se calcula el Índice de Jaccard (IoU) para evaluar el rendimiento.

Ahora una breve descripción del flujo de la ejecución del código:

- Carga la imagen y la convierte a escala de grises.
- Aplica el filtro Sobel para detectar bordes.
- Calcula el GVF para suavizar y mejorar las fuerzas del gradiente.
- Muestra visualmente cada paso del proceso.
- Calcula el IoU para medir qué tan bien se ajusta la segmentación a la referencia.

Este código aplica técnicas de detección de bordes y campo de fuerza de gradiente (GVF) para segmentar una imagen, comparándola con un ground truth de referencia. A continuación, procedemos con su explicación en detalle:

1. Lo primero importamos las librerías que requerimos para ejecutar el código.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
from PIL import Image
from skimage.segmentation import active_contour
```

2. Lo siguiente es cargar y preprocesar la imagen y para esto se carga la imagen con PIL y se convierte a BGR (formato OpenCV). Para luego convertir a escala de grises y se aplica un suavizado Gaussiano para reducir ruido.

```
# Rutas de las imágenes
img_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio
gt_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio N

# Cargar la imagen con PIL
image_pil = Image.open(img_path)
image = cv2.cvtColor(np.array(image_pil), cv2.COLOR_RGB2BGR)

# Convertir a escala de grises y aplicar suavizado
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
image_gray = cv2.GaussianBlur(image_gray, (5, 5), 1.5)
```

3. Ahora se carga y se ejecuta el procesamiento del ground truth y para esto se carga el ground truth desde un archivo .mat y se convierte en una imagen binaria, donde los píxeles relevantes se establecen en 1.

```
# Rutas de las imágenes
img_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio
gt_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio N

# Cargar la imagen con PIL
image_pil = Image.open(img_path)
image = cv2.cvtColor(np.array(image_pil), cv2.COLOR_RGB2BGR)

# Convertir a escala de grises y aplicar suavizado
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
image_gray = cv2.GaussianBlur(image_gray, (5, 5), 1.5)

# Cargar el ground truth
mat_data = sio.loadmat(gt_path)
ground_truth = mat_data['groundTruth'][0, 0]['Segmentation'][0, 0]
ground_truth_binary = (ground_truth > np.min(ground_truth)).astype(np.uint8)
```

4. Se procede entonces con la detección de bordes con Sobel donde se aplica el filtro Sobel en direcciones X e Y para calcular gradientes, se obtiene la magnitud del gradiente combinando ambos resultados, se normaliza el resultado y se convierte a un formato adecuado para el procesamiento posterior.

```
# Aplicar el filtro Sobel
sobel_x = cv2.Sobel(image_gray, cv2.CV_64F, 1, 0, ksize=5)
sobel_y = cv2.Sobel(image_gray, cv2.CV_64F, 0, 1, ksize=5)
sobel_magnitud = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_magnitud = np.uint8(np.clip(sobel_magnitud, 0, 255))
```

5. Ahora se procede con el cálculo del Campo de Fuerza de Gradiente (GVF) el cual es un método que suaviza los gradientes de una imagen para ayudar en la detección de contornos. Y se calcula usando un proceso iterativo con:

- a. Suavizado Gaussiano previo.

- b. Cálculo de gradientes iniciales.
- c. Aplicación del operador Laplaciano para propagar la información de bordes.
- d. Corrección iterativa con un parámetro de regularización μ (controla la difusión del gradiente).

```
# Calcular GVF completo (iterativo)
def compute_gvf(edge_map, mu=0.07, iterations=200):
    edge_map = cv2.GaussianBlur(edge_map, (5, 5), 1.5) # Suavizamos antes de GVF
    edge_map = edge_map.astype(np.float32) / 255.0

    grad_x, grad_y = np.gradient(-edge_map)
    u, v = grad_x.copy(), grad_y.copy()

    for _ in range(iterations):
        laplacian_u = cv2.Laplacian(u, cv2.CV_32F)
        laplacian_v = cv2.Laplacian(v, cv2.CV_32F)

        grad_squared = grad_x**2 + grad_y**2
        grad_squared[grad_squared == 0] = 1e-8 # Evita divisiones por 0

        u += mu * 4 * laplacian_u - (u - grad_x) * grad_squared
        v += mu * 4 * laplacian_v - (v - grad_y) * grad_squared

    gvf_approx = np.sqrt(u**2 + v**2)
    gvf_approx = (gvf_approx - gvf_approx.min()) / (gvf_approx.max() - gvf_approx.min() + 1e-8)

    return gvf_approx, u, v

# Calcular el GVF con la nueva función
gvf_approx, u, v = compute_gvf(sobel_magnitude)
```

6. Procedemos entonces con la visualización de resultados donde se generan 6 imágenes en una figura con Ground truth de referencia, luego la imagen original en color, luego una imagen en escala de grises, la imagen de la Magnitud de Sobel (bordes detectados), el GVF calculado (mapa de fuerza de gradiente) y la superposición de la segmentación con el ground truth.

```
# Mostrar imágenes con mejor distribución
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

axes[0, 0].imshow(ground_truth, cmap='jet')
axes[0, 0].set_title("Ground Truth Original")
axes[0, 0].axis("off")

axes[0, 1].imshow(image, cmap='gray')
axes[0, 1].set_title("Imagen Original")
axes[0, 1].axis("off")

axes[0, 2].imshow(image_gray, cmap='gray')
axes[0, 2].set_title("Imagen en Escala de Grises")
axes[0, 2].axis("off")

axes[1, 0].imshow(sobel_magnitude, cmap='gray')
axes[1, 0].set_title("Magnitud de Sobel")
axes[1, 0].axis("off")

axes[1, 1].imshow(gvf_approx, cmap='magma') # O usa 'inferno', 'viridis', etc.
axes[1, 1].set_title("Campo de Fuerza de Gradiente (GVF)")
axes[1, 1].axis("off")

axes[1, 2].imshow(sobel_magnitude, cmap='gray', alpha=0.5)
axes[1, 2].imshow(ground_truth_binary, cmap='jet', alpha=0.5)
axes[1, 2].set_title("Segmentación vs Ground Truth")
axes[1, 2].axis("off")

plt.tight_layout()
```

7. Procedemos entonces con el cálculo del Índice de Jaccard (IoU) donde se mide qué tan bien la segmentación automática se superpone con el ground truth. Además si las imágenes tienen diferente tamaño, se redimensiona el ground truth, se calcula la intersección y unión de ambas imágenes y por último se muestra el IoU final en la figura.

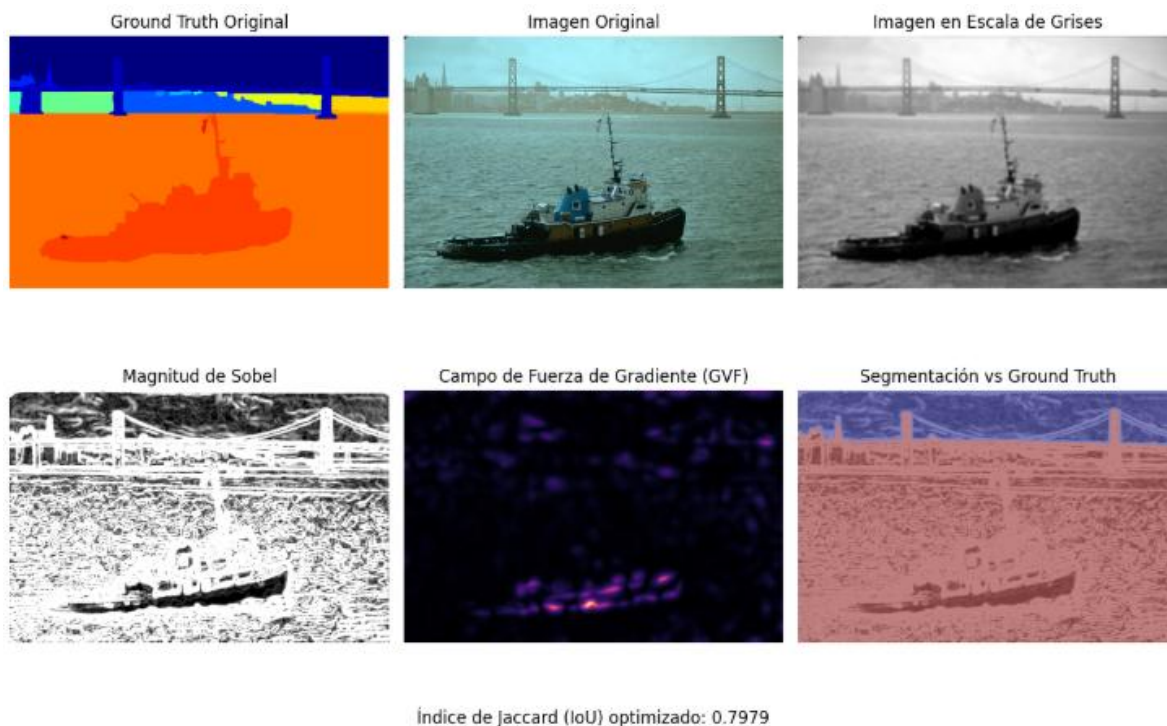
```
# Calcular IoU
def calculate_iou(segmented, ground_truth):
    if segmented.shape != ground_truth.shape:
        ground_truth_resized = cv2.resize(ground_truth, (segmented.shape[1], segmented.shape[0]), interpolation=cv2.INTER_NEAREST)
    else:
        ground_truth_resized = ground_truth
    intersection = np.logical_and(segmented, ground_truth_resized > 0)
    union = np.logical_or(segmented, ground_truth_resized > 0)
    return np.sum(intersection) / np.sum(union) if np.sum(union) > 0 else 0

iou = calculate_iou(sobel_magnitude, ground_truth_binary)

# Agregar el texto del IoU en la parte inferior de la figura principal
fig.text(0.5, 0.02, f"Índice de Jaccard (IoU) optimizado: {iou:.4f}", ha="center", fontsize=12)

# Mostrar la figura completa
plt.show()
```

8. A continuación, damos a conocer los resultados de la consola de Jupyter Notebook



METODO WATERSHED (CUENCAS HIDROGRAFICAS)

Como una introducción a este método debemos decir que El algoritmo Watershed es una técnica de segmentación utilizada en visión por computadora para dividir una imagen en diferentes regiones basadas en su topografía. Este método interpreta una imagen en escala de grises como una superficie tridimensional, donde los niveles de intensidad representan alturas. Luego, trata de encontrar cuencas y crestas para segmentar objetos dentro de la imagen. A continuación, el respectivo desarrollo y la explicación de este:

1. Como primera medida procedemos a cargar las librerías que requerimos para el código:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
from PIL import Image
```

2. Ahora procedemos con la carga y preprocesamiento de la imagen donde se carga la imagen con PIL y se convierte a formato BGR para usar con OpenCV para luego convertirla a escala de grises y aplicar un suavizado Gaussiano para reducir el ruido.

```
# Rutas de las imágenes
img_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio N
gt_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio N

# Cargar imagen
image_pil = Image.open(img_path)
image = cv2.cvtColor(np.array(image_pil), cv2.COLOR_RGB2BGR)

# Cargar el ground truth
mat_data = sio.loadmat(gt_path)
ground_truth = mat_data['groundTruth'][0, 0][['Segmentation']][0, 0]

# Convertir a escala de grises
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
image_gray_blur = cv2.GaussianBlur(image_gray, (3, 3), 0)
```

3. Entonces procedemos con la carga y procesamiento del Ground Truth donde se carga el ground truth desde un archivo .mat y se convierte en una imagen binaria, donde los píxeles relevantes se establecen en 1.

```
# Cargar el ground truth
mat_data = sio.loadmat(gt_path)
ground_truth = mat_data['groundTruth'][0, 0][['Segmentation']][0, 0]
```

4. Ahora procedemos con la detección de bordes con Sobel en donde aplicamos el filtro Sobel en direcciones X e Y para calcular los gradientes. Luego obtenemos la magnitud del gradiente combinando ambos resultados y normalizamos y convertimos a un formato adecuado para el procesamiento posterior.

```
# Aplicar filtro Sobel
sobel_x = cv2.Sobel(image_gray_blur, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(image_gray_blur, cv2.CV_64F, 0, 1, ksize=3)

sobel_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_magnitude = cv2.convertScaleAbs(sobel_magnitude)
```

5. Entonces desarrollamos la generación de marcadores para Watershed donde se realiza un thresholding sobre la imagen de Sobel para obtener los marcadores iniciales y se agregan bordes artificiales como marcadores en los extremos de la imagen para mejorar la segmentación.

```
# Crear marcadores automáticos con thresholding
_, markers_auto = cv2.threshold(sobel_magnitude, 30, 255, cv2.THRESH_BINARY)
markers = np.zeros_like(image_gray, dtype=np.int32)
markers[markers_auto == 255] = 2 # Áreas detectadas con Sobel

# Agregar bordes como marcadores
h, w = image_gray.shape
markers[0:10, :] = 1 # Bordes superiores
markers[h-10:h, :] = 1 # Bordes inferiores
markers[:, 0:10] = 1 # Bordes izquierdos
markers[:, w-10:w] = 1 # Bordes derechos
```

6. Ahora procedemos con la aplicación del algoritmo Watershed, y para esto se convierte la imagen de escala de grises a RGB para el procesamiento, luego ejecutamos Watershed, que asigna etiquetas a las diferentes regiones y se obtiene la segmentación final y se convierte en una imagen binaria.

```
# Aplicar Watershed
image_rgb = cv2.cvtColor(image_gray, cv2.COLOR_GRAY2BGR)
markers = cv2.watershed(image_rgb, markers)
segmented = markers.copy()
segmented[segmented == -1] = 0
segmented_binary = (segmented > 1).astype(np.uint8)
```

7. Entonces ejecutamos el postprocesamiento y suavizado, donde se aplica una dilatación para suavizar los bordes de la segmentación y mejorar la apariencia final.

```
# Suavizar bordes con dilatación
kernel = np.ones((3, 3), np.uint8)
segmented_binary = cv2.dilate(segmented_binary, kernel, iterations=2)
```

8. Procedemos entonces con el cálculo del Índice de Jaccard (IoU) donde, se compara la segmentación generada con el ground truth, ahora si las imágenes tienen diferente tamaño, se redimensiona el ground truth. Adicionalmente, se calcula la intersección y la unión de ambas imágenes y se obtiene el IoU final, que mide la superposición de la segmentación con el ground truth.

```
# Calcular IoU
def calculate_iou(segmented, ground_truth):
    if segmented.shape != ground_truth.shape:
        ground_truth_resized = cv2.resize(ground_truth, (segmented.shape[1], segmented.shape[0]), interpolation=cv2.INTER_NEAREST)
    else:
        ground_truth_resized = ground_truth
    intersection = np.logical_and(segmented, ground_truth_resized > 0)
    union = np.logical_or(segmented, ground_truth_resized > 0)
    iou_score = np.sum(intersection) / np.sum(union)
    return iou_score

ground_truth_binary = (ground_truth > 0).astype(np.uint8)
iou = calculate_iou(segmented_binary, ground_truth_binary)
```

9. Procedemos con la visualización de resultados, donde se generan 4 imágenes en una figura que son:
- La imagen en escala de grises.
 - La imagen con la magnitud de Sobel (bordes detectados).
 - La segmentación final con Watershed.
 - El Ground truth de referencia.
 - Y finalmente se muestra el valor del IoU en la parte inferior de la figura.

```
# Visualizar todas las imágenes
fig, axes = plt.subplots(1, 4, figsize=(15, 5))
axes[0].imshow(image_gray, cmap='gray')
axes[0].axis('off')
axes[0].set_title("Imagen Original (Grises)")

axes[1].imshow(sobel_magnitude, cmap='gray')
axes[1].axis('off')
axes[1].set_title("Filtro Sobel")

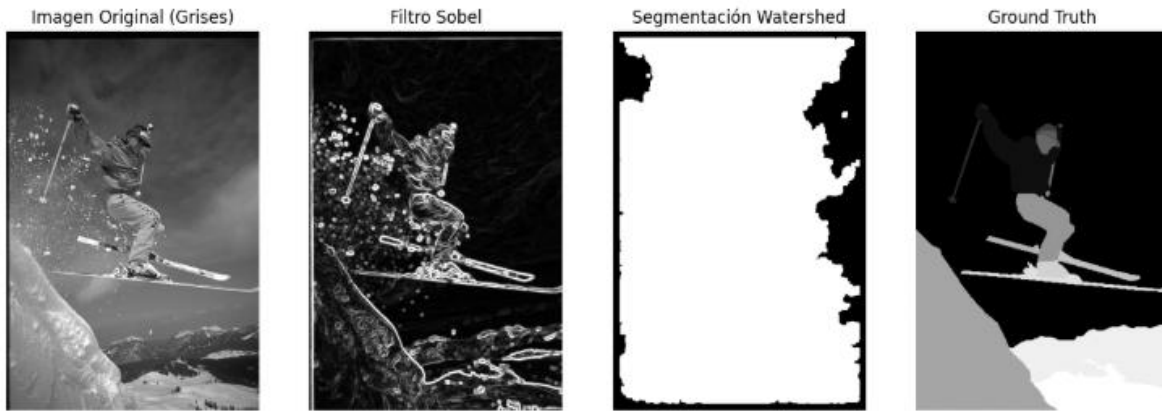
axes[2].imshow(segmented_binary, cmap='gray')
axes[2].axis('off')
axes[2].set_title("Segmentación Watershed")

axes[3].imshow(ground_truth, cmap='gray')
axes[3].axis('off')
axes[3].set_title("Ground Truth")

# Agregar el texto del IoU en la parte inferior del gráfico
plt.figtext(0.5, 0.02, f"Índice de Jaccard (IoU) para Watershed con Sobel: {iou:.4f}", ha="center", fontsize=12)

plt.show()
```

10. A continuación, los resultados de la consola de Jupyter Notebook:



Índice de Jaccard (IoU) para Watershed con Sobel: 0.8195

METODO CRECIMIENTO DE REGIONES BASADO EN GRAFOS (GRAPH-BASED REGION GROWING)

1. Lo primero es que cargamos las librerías como en los anteriores métodos para que podamos ejecutar el código.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import os
from PIL import Image
from skimage.segmentation import felzenszwalb
```

2. Luego procedemos con la carga y preprocesamiento de la imagen, donde se carga la imagen con PIL, se convierte a formato BGR para su uso en OpenCV y se convierte a escala de grises para su visualización.

```
# Rutas de las imágenes
img_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio 1\Imagenes\img.jpg"
gt_path = r"C:\Users\gran_\Documents\Estudio\Especialización Inteligencia Artificial\Segundo semestre\Percepción Computacional\Laboratorio\Laboratorio 1\Imagenes\gt.mat"

# Verificar si los archivos existen
if not os.path.exists(img_path):
    print(f"Error: El archivo {img_path} no existe en la ruta especificada.")
if not os.path.exists(gt_path):
    print(f"Error: El archivo {gt_path} no existe en la ruta especificada.")

# Cargar la imagen con PIL
try:
    image_pil = Image.open(img_path)
    image = cv2.cvtColor(np.array(image_pil), cv2.COLOR_RGB2BGR)
except Exception as e:
    print(f"Error al cargar la imagen con PIL: {e}")
    image = None
```

3. Ahora ejecutamos el proceso de carga y procesamiento del Ground Truth y para esto se carga el ground truth desde un archivo .mat. Luego se extrae la primera segmentación proporcionada por humanos y se convierte en una imagen binaria, donde los píxeles relevantes se establecen en 1.

```
# Cargar el ground truth desde el archivo .mat
mat_data = sio.loadmat(gt_path)
ground_truth = mat_data['groundTruth'][0, 0]['Segmentation'][0, 0] # Primera segmentación humana
```

4. Ahora procedemos con la verificación de datos donde se valida que la imagen y el ground truth se hayan cargado correctamente y si no se encuentran los archivos, se muestra un mensaje de error.

```
# Validar carga de imágenes
if image is None:
    print(f"Error: No se pudo cargar la imagen en {img_path}")
if ground_truth is None:
    print(f"Error: No se pudo cargar la ground truth en {gt_path}")
```

5. Entonces procedemos con la aplicación de segmentación basada en grafos (Felzenszwalb) donde aplicamos el algoritmo Felzenszwalb para segmentar la imagen en regiones y se utiliza una escala de 100, una suavización sigma de 0.5 y un tamaño mínimo de 50 píxeles por región.

```
# Si ambas imágenes se cargaron, continuar
if image is not None and ground_truth is not None:
    # Convertir a escala de grises para visualización
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Aplicar segmentación basada en grafos (Felzenszwalb)
    segmented = felzenszwalb(image, scale=100, sigma=0.5, min_size=50)
```

6. Entonces procedemos con la binarización de la segmentación en donde se convierte la segmentación obtenida en una imagen binaria y se consideran las regiones con valores mayores a 0 como objetos segmentados.

```
# Binarizar la segmentación (tomar regiones > 0 como objetos)
segmented_binary = (segmented > 0).astype(np.uint8)
```

7. Ahora procedemos con el cálculo del Índice de Jaccard (IoU) en donde comparamos la segmentación obtenida con el ground truth y si las imágenes tienen diferente tamaño, se redimensiona el ground truth para coincidir con la segmentación para luego calcular la intersección y la unión de ambas imágenes para obtener el IoU final, que mide la precisión de la segmentación.

```
# Calcular IoU
def calculate_iou(segmented, ground_truth):
    if segmented.shape != ground_truth.shape:
        ground_truth_resized = cv2.resize(ground_truth, (segmented.shape[1], segmented.shape[0]), interpolation=cv2.INTER_NEAREST)
    else:
        ground_truth_resized = ground_truth
    intersection = np.logical_and(segmented, ground_truth_resized > 0)
    union = np.logical_or(segmented, ground_truth_resized > 0)
    iou_score = np.sum(intersection) / np.sum(union)
    return iou_score

ground_truth_binary = (ground_truth > 0).astype(np.uint8)
iou = calculate_iou(segmented_binary, ground_truth_binary)
```

8. Entonces procedemos con la visualización de resultados donde se generan 3 imágenes en una figura:
- Imagen original en escala de grises.
 - Imagen segmentada con el método basado en grafos.
 - Ground truth de referencia.
 - Y se muestra el valor del IoU en la parte inferior de la figura.


```
# Visualizar resultados: Imagen Original, Segmentación Grafos y Ground Truth
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(image_gray, cmap='gray')
axes[0].axis('off')
axes[0].set_title("Imagen Original (Grises)")

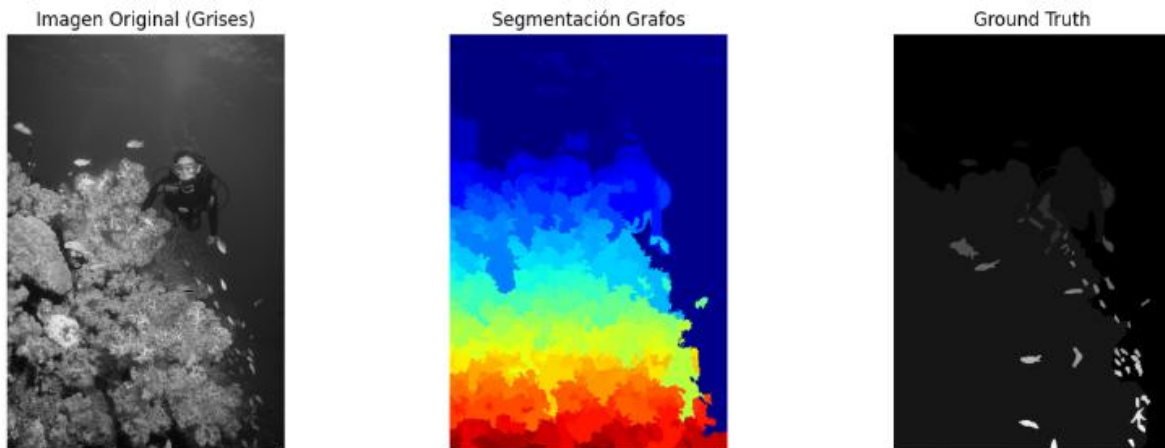
axes[1].imshow(segmented, cmap='jet')
axes[1].axis('off')
axes[1].set_title("Segmentación Grafos")

axes[2].imshow(ground_truth, cmap='gray')
axes[2].axis('off')
axes[2].set_title("Ground Truth")

# Agregar el IoU como texto en la parte inferior de la figura
plt.figtext(0.5, 0.02, f"Índice de Jaccard (IoU) para Crecimiento de Regiones basado en Grafos (Graph-Based Region Growing): {iou:.4f}", ha="center")

plt.show()
```

9. A continuación, los resultados de la consola de Jupyter Notebook:



Índice de Jaccard (IoU) para Crecimiento de Regiones basado en Grafos (Graph-Based Region Growing): 0.8598

CONCLUSIONES

A lo largo de esta actividad, se implementaron y analizaron diferentes técnicas de segmentación de imágenes, con un enfoque particular en el Crecimiento de Regiones basado en Grafos mediante el algoritmo de Felzenszwalb. Este método permitió descomponer la imagen en regiones coherentes basadas en la similitud de los píxeles, facilitando la identificación de objetos y estructuras dentro de la imagen.

El método de Felzenszwalb, basado en una representación gráfica de la imagen, logró generar una segmentación en la que los píxeles fueron agrupados en regiones de acuerdo con sus características de color e intensidad. Su funcionamiento se basa en la construcción de un grafo ponderado, donde los nodos representan píxeles y los bordes indican la similitud entre ellos. Posteriormente, se emplea un criterio de homogeneidad para definir las regiones segmentadas.

La evaluación del método se realizó mediante la comparación con el ground truth, una segmentación de referencia anotada manualmente. Para medir la precisión del resultado, se utilizó el Índice de Jaccard (IoU), el cual cuantifica la superposición entre la segmentación obtenida y la segmentación de referencia. Se encontró que el valor del IoU dependía de factores como:

- Escala utilizada en el algoritmo: Afecta la granularidad de la segmentación, determinando si se generan muchas regiones pequeñas o pocas regiones grandes.
- Tamaño mínimo de las regiones: Define el tamaño de los segmentos resultantes, lo que influye en el nivel de detalle.
- Parámetro sigma (suavizado de la imagen antes de la segmentación): Un mayor suavizado reduce el ruido, pero puede perder detalles finos.

En comparación con otras técnicas como Watershed o K-Means, el método basado en grafos presenta ciertas ventajas y desventajas:

Ventajas:

- No requiere definir el número de regiones de antemano.
- Se adapta mejor a variaciones en la textura y los bordes de la imagen.
- Es eficiente y puede manejar imágenes de gran tamaño sin un alto costo computacional.

Desventajas:

- La segmentación puede ser sensible a los parámetros seleccionados, lo que dificulta su ajuste óptimo.
- En imágenes con alto nivel de ruido o variaciones de iluminación, el rendimiento puede disminuir.
- Puede fragmentar regiones homogéneas en múltiples partes si los parámetros no están bien configurados.

Ahora para mejorar los resultados obtenidos, se pueden explorar diversas estrategias:

- Optimización de los parámetros: Aplicar técnicas de optimización para encontrar los valores ideales de escala, sigma y tamaño mínimo de región.
- Combinación con otras técnicas: Integrar el método de Felzenszwalb con técnicas de postprocesamiento como la morfología matemática o refinamiento mediante Redes Neuronales Convolucionales (CNNs).
- Implementación en diferentes tipos de imágenes: Evaluar su desempeño en imágenes médicas, satelitales o industriales, donde la segmentación precisa es crucial.
- Uso de aprendizaje automático: Aplicar modelos de Deep Learning que aprendan a ajustar automáticamente los parámetros de segmentación.

Como reflexión final nos permitimos indicar que la segmentación de imágenes es un proceso fundamental en visión por computadora y procesamiento de imágenes, con aplicaciones en campos como la medicina, la seguridad y la robótica. Si bien el Crecimiento de Regiones basado en Grafos es una técnica potente y flexible, su desempeño depende en gran medida de la configuración de parámetros y la naturaleza de la imagen. En el futuro, su combinación con inteligencia artificial y técnicas de aprendizaje automático permitirá obtener segmentaciones más precisas y robustas.

En conclusión, este laboratorio nos permitió comprender el potencial y las limitaciones de este método, resaltando la importancia de ajustar correctamente sus parámetros y considerar alternativas híbridas para mejorar su precisión.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en la búsqueda de información:

- Tema 6. Procesamiento de imagen. Operaciones elementales. Tema 7. Procesamiento de imagen. Operaciones espaciales. Tema 8. Procesamiento de imagen. Morfología matemática. Tema 9. Procesamiento de señales. Filtrado y análisis en frecuencia. Tema 10. Procesamiento de imagen. Crecimiento de regiones.
- Clases virtuales con el profesor Ing. Magda Del Pilar Santa Fajardo.
- Herramienta Jupiter Notebook - Python.